Lumen

A programming language where every number knows its unit

```
let v: km/h = 10 m/s;
print(v);
```

Review 2: design and architecture, 4 of 7 phases built

Compiler Design (BCSE307L)

Sanjay Suresh 24BCE2685
Sugesh Kumar 24BCE2687
Noel Ninan 24BCE2737
Advaith Sivakumar 24BCE2715
Ajay Kumar 24BCE2721

The problem we are solving

1999 · Mars Climate Orbiter

×4.45

Pound-force seconds were read as newton-seconds. The spacecraft was lost: $327.6 million.

1983 · Air Canada "Gimli Glider"

1.77 ≠ 0.803

Pounds per litre were used as kg per litre. The plane ran out of fuel at 41,000 ft.

To a normal compiler both numbers are just float, so nothing notices.

Our goal: a language whose compiler catches unit mistakes before the program runs, and costs nothing at run time.

Objectives and scope

What we will build

- A small language where the unit is part of a number's type
- A seven-phase compiler for it, in plain Python
- Exact unit conversion, and clear errors with a help line
- Units erased after checking: no run-time cost
- Tests for every phase

What we leave out

- Temperatures like °C (they need offsets)
- Telling torque from energy (same dimension)
- Checking across several programs
- Native machine code: we target our own VM

Language design

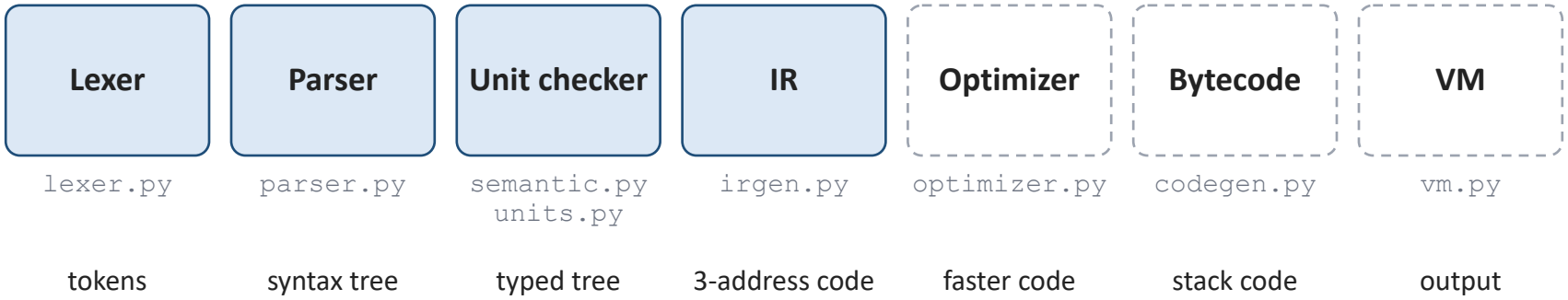
```
unit min = 60 s;
unit h   = 3600 s;

fn pace(d: km, t: min) -> km/h {
    return d / t;
}

fn main() {
    let d = 42.195 km;
    let t = 3 h + 30 min;
    print(pace(d, t));
    print(d in m);
}
```

- A number followed by a unit is a quantity: 42.195 km
- Units are declared with unit; unit INR; makes a new base unit
- Types can be units: km, N*s, km/h
- in converts explicitly
- Ordinary parts: let, fn, if, while, return, print
- Types: int, float, bool, string and units

Architecture: seven phases



Solid blue: built and tested. Dashed: designed, to be built for Review 3.

Front end (lexer, parser, checker) understands the program and its units. The middle (IR, optimizer) simplifies it. The back end (bytecode, VM) runs it. Units exist only in the front end; from the IR onwards a quantity is a plain number.

What each phase hands to the next

Phase	Input	Output	Key data structure
Lexer	source text	tokens	`Token(kind, text, line, col)`
Parser	tokens	syntax tree	`Node classes: Binary, Let, Call...`
Unit checker	syntax tree	typed tree + errors	`Type(kind, dimension, scale)`
IR	typed tree	3-address code	`Instr(op, dst, a, b)`
Optimizer	IR	smaller IR	same IR, rewritten
Bytecode	IR	stack code	`BC(op, arg)`
VM	stack code	program output	`Frame(slots, stack, ip)`

Errors from every phase use one shared record: Diagnostic (severity, message, line, column, help).

Grammar and parsing approach

```
program   = { unitDecl | function } EOF
unitDecl  = "unit" IDENT [ "=" NUMBER unitExpr ] ";"
function  = "fn" IDENT "(" [params] ")" block
statement = let | assign | if | while
          | return | print | block
expr      = logicOr [ "in" unitExpr ]
term      = factor { ("+"|"-") factor }
factor    = unary { ("*"|"/"|"%") unary }
primary   = NUMBER [unit] | IDENT | "(" expr ")"
```

- Hand-written recursive descent: one function per rule
- LL(1): one token of lookahead
- Precedence from layering the rules
- Two unit rules: a name after a number is a unit; a unit continues over * and / only if the next name is a known unit
- Panic-mode error recovery

Unit type system

Every number has three parts: kind, dimension, scale

Unit	Dimension (powers of m, kg, s)	Scale
`m`	length m^1	1
`km`	length m^1	1000
`km/h`	speed $m^1\ s^{-1}$	5/18
`N`	force $kg^1\ m^1\ s^{-2}$	1

Operator	Rule
`+ - <`	dimensions equal
`*`	add the powers
`/`	subtract the powers
`in`	same dimension

Scales are exact fractions, so conversions never drift. Conversion changes the scale only, never the dimension, and a plain number never becomes a quantity (except 0).

Error handling design

- Errors are collected, not thrown: one run shows many
- A bad expression gets an error type, so there is no cascade
- Every error has a line, a caret and usually a help line
- Lexical errors are shown alone; syntax errors stop before the checker
- Never a Python crash on a user's mistake

```
error: dimension mismatch: cannot add
  time (s) to length (m)
   10 |   let bad = race + t;
      |                  ^
   = help: '+' needs the same dimension
     on both sides.
```

Exit codes

```
0 ran fine    1 compile error
2 bad file    3 runtime error
```

Progress so far: about 60%

4 of 7 phases built

Phase	Status	What works today
1 Lexer	**Done**	tokens with line and column, exact numbers, all lexical errors
2 Parser	**Done**	full grammar, syntax tree, error recovery
3 Unit checker	**Done**	dimension algebra, conversions, scope, types, warnings
4 IR	**Done**	three-address code, short-circuit jumps, loops
5 Optimizer	**Planned**	design done (folding, fusion, dead code)
6 Bytecode	**Planned**	instruction set designed
7 VM	**Planned**	frame and error model designed

What runs today

`python -m lumen run phases_demo.lum --ir`

```
func main():
    d = 90.0
    t = 1.5
    t1 = d / t
    v = t1
    print v "km/h"
    t2 = v * 0.2777777777777778
                    ; convert
    print t2 "m/s"
    return
```

The unit conversion is already a plain multiplication here.

Mars bug, rejected by the checker

```
program.lum:13:23: error: argument 1 of 'update_trajectory':
expected impulse (N*s), found a plain number (float)
   13 |     update_trajectory(from_ground_file);  // Lumen
refuses to compile this
      |                       ^~~~~~~~~~~~~~
      = help: if the number really is in N*s, say so:
`from_ground_file * 1 N*s`; or write the literal with its unit
(e.g. `4 N*s`)

1 error generated.
```

exit code 1 compile error 5 ms

A bare 12.5 where N*s is needed is a compile error with a fix suggested.

Testing so far

102

automated tests pass for phases 1 to 3

Module	Tests
Unit checker	63
Parser	20
Lexer	12
Unit algebra	7

- Each phase is tested on its own
- Error messages are tested, not only success cases
- Example programs (Mars, Gimli, dosing, currency) are checked by hand

Planned for Review 3: golden outputs for every example, and a test that the optimizer never changes a program's output.

Remaining work for Review 3

Work	What it involves	Owner
Optimizer	constant folding and propagation, conversion fusion, dead code, branch pruning	Noel
Bytecode generator	stack instruction set, slots and labels	Ajay
Virtual machine	frames, runtime errors, step and depth limits	Ajay
Test suite	golden outputs, optimizer-on versus off comparison	Ajay, Noel
Demo and docs	web playground, examples, viva notes	Ajay, Sanjay

Risk: optimizer bugs could change a program's meaning. Plan: share one arithmetic function between the optimizer and the VM, and never fold a division that can fail.

Team and responsibilities

Member	Register no.	Part
Sanjay Suresh	24BCE2685	Lexer, error messages, command line
Advaith Sivakumar	24BCE2715	Parser and syntax tree
Sugesh Kumar	24BCE2687	Unit system and unit checker
Noel Ninan	24BCE2737	Intermediate code and optimizer
Ajay Kumar	24BCE2721	Bytecode, VM, tests, playground

Each of us can explain every phase; the split decides who goes deepest.